REAL ESTATE ERP SYSTEM

Complete Pre-Development Technical Blueprint Suite (Public Portal, Off-Plan, Sales & Leasing)

AUTHOR & ROLE
Chief Technology Officer

ARCHITECTURE TARGET
Decoupled Multi-Tenant ERP & Web Portal

DOCUMENT VERSION
Version 2.0.0-PROD (July 2026)

DOCUMENT 01

Software Requirements Specification (SRS)

1. Executive Summary & System Scope

The Real Estate ERP System is an enterprise-grade multi-tenant platform managing off-plan sales (incomplete construction), completed property sales, long-term rentals/leasing, public-facing portal discovery, construction milestone tracking, customer CRM, and double-entry multi-currency general ledger accounting. The system provides atomic inventory protection against double booking and dynamic, metadata-driven administration.

2. User Roles & Access Control Framework

Role Title	Responsibilities	System Scope
System Admin	Global configuration, RBAC, tenant isolation, database recovery, audit inspection.	Full system operational access across all modules.
Public Visitor	Search listings, view interactive floor plans, reserve off-plan/complete units, apply for lease.	Public-facing web portal only.
Sales Manager	Manage sales teams, approve reservation overrides, unit transfers, contract cancellations.	CRM, Sales, Inventory, and Contract Lifecycle modules.
Buyer / Tenant	Track payment schedules, make online portal payments, download tax invoices and statements.	Customer Self-Service Portal.
Project Manager	Track construction milestones, update stage %, upload site photos and inspection logs.	Construction Management & Site Vault.
Finance & Accounting	General ledger, AR/AP, bank reconciliations, tax (VAT/WHT/Stamp Duty), fixed assets.	ERP Financial Core, Banking & Accounting.
Petty Cash Custodian	Manage local site operational floats, disburse petty cash, submit vouchers and receipts.	Dedicated Petty Cash sub-module.

3. Comprehensive Functional Requirements

3.1 Property & Project Configuration

- **FR-PRJ-01 (Project & Phase Hierarchy):** Create new projects structured into multiple blocks, phases, or master-planned plots. Completed projects can be archived in a read-only state without data deletion.
- **FR-PRJ-02 (Unit Numbering & Bulk Import):** Supports flexible unit numbering schemas (`Block/Floor/Unit/Plot`). Allows importing unit records via Excel (`.xlsx`) templates.
- **FR-PRJ-03 (Multi-Currency Support):** Stores unit pricing in dual base currencies (USD and KES) with live or locked contract-level exchange rates.
- **FR-PRJ-04 (Rich Unit Metadata & Pricing Updates):** Units store floor plans, site photos, unit size, parking slots, storage units, balconies, and status (`AVAILABLE` , `RESERVED` , `SOLD` , `RENTED` , `BLOCKED`). Selling prices can be revised with audit trail logs.

3.2 Customer (CRM) & Ownership Models

- **FR-CRM-01 (Customer Vault):** Stores personal details, uploaded national IDs, passports, KRA PIN certificates, executed agreements, and Next of Kin / Nominee details.

- **FR-CRM-02 (Complex Ownership Patterns):**
  - *One-to-Many:* Single client can own or lease multiple units across different projects.
  - *Many-to-One:* Single unit can be co-owned by multiple clients with specified share percentages.
- **FR-CRM-03 (Ownership Transfer):** Transfer ownership from one client to another while maintaining complete historical ownership logs.

3.3 Sales, Off-Plan Reservations & Allocation

- **FR-SAL-01 (Anti-Double Booking Engine):** Atomic database row-level locking guarantees zero double bookings across the public website and back-office sales interface.
- **FR-SAL-02 (Overrides & Release):** Management users can override reservations or cancel bookings, which automatically returns units to `AVAILABLE` status.
- **FR-SAL-03 (Payment Plans & Auto Contracts):** Supports standard, milestone-driven (off-plan), or custom payment plans, and generates auto-filled sale contracts and offer letters.

3.4 Installments, Collections & Accounts Receivable

- **FR-AR-01 (Dynamic Payment Schedules):** Auto-generates installment schedules with manual editing or milestone-triggered due dates. Calculates outstanding balance and percentage paid.
- **FR-AR-02 (Invoicing & Receipts):** Supports bulk invoicing, automated PDF email invoices, partial payments, allocation to deposit vs. installment, receipt printing/reprinting/cancelling, and customer statements with AR aging reports.

3.5 Contract Lifecycle, Unit Swaps & Reallocations

- **FR-CNT-01 (Unit Swaps / Transfers):** Customers can switch units. Payments automatically transfer to the new unit, price differentials (refund or extra payment) are calculated, and the old unit is released back to available inventory.
- **FR-CNT-02 (Sales Cancellations & Refunds):** Calculates cancellation fees and net refunds. Refunds require strict approval flows and appear on customer statements.
- **FR-CNT-03 (Payment Reallocation):** Misassigned payments can be reallocated across customers, units, or invoices with an immutable audit trail.

3.6 Accounts Payable, Petty Cash & Construction Management

- **FR-AP-01 (Accounts Payable):** Attach supplier invoices, stage payments, reconcile supplier statements, and link expenses to specific projects/units.
- **FR-PC-01 (Petty Cash):** Dedicated multi-box petty cash module with float custodians, printable vouchers, receipt attachments, and reconciliation routines.
- **FR-CON-01 (Construction Progress):** Tracks off-plan construction stages, progress percentages, site photo uploads, and engineer inspection reports.

4. Security, Accounting & System Constraints

- **Financial General Ledger:** Double-entry bookkeeping with manual/reversing journals, multi-bank auto-reconciliation, fixed asset depreciation, period locking, and full tax engines (VAT, WHT, Stamp Duty).
- **Security & Audit:** Role-based access control (RBAC), project-level data isolation, 2FA authentication, immutable audit log, and soft-delete record recovery.

DOCUMENT 02

Software Design Document (SDD)

1. Technical Stack & Cloud Architecture

- **Frontend Framework:** React 18+ (TypeScript), Vite, TailwindCSS, React Hook Form, Zod.
- **Backend Framework:** NestJS (Node.js/TypeScript) implementing modular domain-driven architecture.

- **Database Layer:** PostgreSQL 16+ featuring Row-Level Security (RLS) for tenant/project isolation.
- **Caching & Queues:** Redis Enterprise for session locks, listing caches, and BullMQ background job processing.
- **Object Storage:** AWS S3 (KMS Encrypted) for customer passports, PINs, vendor invoices, and site photos.

2. Database Schema (PostgreSQL DDL)

```
-- 1. PROJECTS & UNITS
CREATE TYPE unit_status_enum AS ENUM ('AVAILABLE', 'RESERVED', 'SOLD', 'RENTED', 'BLOCKED');

CREATE TABLE projects (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    code VARCHAR(30) UNIQUE NOT NULL,
    name VARCHAR(150) NOT NULL,
    location VARCHAR(150),
    is_archived BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE TABLE project_blocks (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID REFERENCES projects(id) ON DELETE CASCADE,
    block_name VARCHAR(50) NOT NULL,
    total_floors INT NOT NULL
);

CREATE TABLE units (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    block_id UUID REFERENCES project_blocks(id) ON DELETE RESTRICT,
    unit_number VARCHAR(30) NOT NULL,
    floor_number INT NOT NULL,
    size_sqm NUMERIC(10,2) NOT NULL,
    price_kes NUMERIC(14,2) NOT NULL,
    price_usd NUMERIC(14,2) NOT NULL,
    status unit_status_enum DEFAULT 'AVAILABLE',
    parking_slots INT DEFAULT 0,
    has_balcony BOOLEAN DEFAULT FALSE,
    has_store BOOLEAN DEFAULT FALSE,
    floor_plan_url TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    version INT DEFAULT 1
);

-- 2. CRM & MULTI-OWNERSHIP
CREATE TABLE customers (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    phone VARCHAR(30) NOT NULL,
    national_id_passport VARCHAR(50) NOT NULL,
    kra_pin VARCHAR(50),
    next_of_kin_json JSONB DEFAULT '{}'::jsonb,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE TABLE unit_ownership (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    unit_id UUID REFERENCES units(id) ON DELETE RESTRICT,
    customer_id UUID REFERENCES customers(id) ON DELETE RESTRICT,
    ownership_percentage NUMERIC(5,2) DEFAULT 100.00,
    is_primary_owner BOOLEAN DEFAULT TRUE,
    acquired_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    transferred_at TIMESTAMP WITH TIME ZONE
);

-- 3. CONTRACTS & REALLOCATIONS
CREATE TABLE sales_contracts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    contract_number VARCHAR(50) UNIQUE NOT NULL,
    unit_id UUID REFERENCES units(id) ON DELETE RESTRICT,
    primary_customer_id UUID REFERENCES customers(id) ON DELETE RESTRICT,
    currency VARCHAR(3) DEFAULT 'KES',
```

```
    total_agreed_price NUMERIC(14,2) NOT NULL,
    contract_status VARCHAR(30) DEFAULT 'ACTIVE',
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE TABLE customer_payments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    receipt_number VARCHAR(50) UNIQUE NOT NULL,
    contract_id UUID REFERENCES sales_contracts(id) ON DELETE RESTRICT,
    amount_paid NUMERIC(14,2) NOT NULL,
    currency VARCHAR(3) NOT NULL,
    payment_method VARCHAR(30) NOT NULL,
    transaction_reference VARCHAR(100) UNIQUE NOT NULL,
    payment_date TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE TABLE payment_reallocations_audit (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    payment_id UUID REFERENCES customer_payments(id),
    source_contract_id UUID,
    destination_contract_id UUID,
    reallocated_amount NUMERIC(14,2) NOT NULL,
    reason TEXT NOT NULL,
    reallocated_by UUID NOT NULL,
    timestamp TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

3. Unit Swap & Transactional Payment Transfer Engine

```typescript
// src/modules/sales/unit-transfer.service.ts
import { Injectable, BadRequestException } from '@nestjs/common';
import { DataSource, EntityManager } from 'typeorm';
import { SalesContract } from './entities/sales-contract.entity';
import { Unit, UnitStatus } from '../inventory/entities/unit.entity';
import { CustomerPayment } from '../finance/entities/customer-payment.entity';
import { PaymentReallocationAudit } from '../finance/entities/reallocation-audit.entity';

@Injectable()
export class UnitTransferService {
  constructor(private readonly dataSource: DataSource) {}

  async executeUnitTransfer(
    currentContractId: string,
    targetUnitId: string,
    requestedByUserId: string,
    transferReason: string
  ): Promise<{ newContractId: string; priceDifference: number }> {
    return await this.dataSource.transaction(async (manager: EntityManager) => {
      const currentContract = await manager.findOne(SalesContract, {
        where: { id: currentContractId, contractStatus: 'ACTIVE' },
        relations: ['unit', 'payments'],
        lock: { mode: 'pessimistic_write' }
      });

      if (!currentContract) throw new BadRequestException('Active source contract not found.');

      const targetUnit = await manager.findOne(Unit, {
        where: { id: targetUnitId },
        lock: { mode: 'pessimistic_write' }
      });

      if (!targetUnit || targetUnit.status !== UnitStatus.AVAILABLE) {
        throw new BadRequestException('Target unit unavailable.');
      }

      const totalPaidOnOldUnit = currentContract.payments.reduce((sum, p) => sum + Number(p.amountPaid), 0);
      const priceDifference = Number(targetUnit.priceKes) - Number(currentContract.totalAgreedPrice);

      // Release old unit & reserve new unit
      currentContract.unit.status = UnitStatus.AVAILABLE;
      await manager.save(Unit, currentContract.unit);

      targetUnit.status = UnitStatus.SOLD;
      await manager.save(Unit, targetUnit);

      // Create new contract & transfer payments
      currentContract.contractStatus = 'SWAPPED';
      await manager.save(SalesContract, currentContract);

      const newContract = manager.create(SalesContract, {
        contractNumber: `${currentContract.contractNumber}-SWAP`,
        unit: targetUnit,
        primaryCustomer: currentContract.primaryCustomer,
        currency: currentContract.currency,
        totalAgreedPrice: targetUnit.priceKes,
        contractStatus: 'ACTIVE'
      });
      const savedNewContract = await manager.save(SalesContract, newContract);

      for (const payment of currentContract.payments) {
        payment.contract = savedNewContract;
        await manager.save(CustomerPayment, payment);

        const audit = manager.create(PaymentReallocationAudit, {
          payment: payment,
```

```
          sourceContractId: currentContract.id,
          destinationContractId: savedNewContract.id,
          reallocatedAmount: payment.amountPaid,
          reason: `Automated Unit Swap: ${transferReason}`,
          reallocatedBy: requestedByUserId
        });
        await manager.save(PaymentReallocationAudit, audit);
      }

      return { newContractId: savedNewContract.id, priceDifference };
    });
  }
}
```

DOCUMENT 03

Project Plan & Statement of Work (SOW)

1. 12-Week Milestone Delivery Roadmap

Milestone	Phase Title	Timeline	Core Scope & Key Deliverables
M1	Architecture & Multi-Currency GL	Weeks 1–2	PostgreSQL database schema, multi-tenant RLS, auth 2FA, multi-currency GL backbone.
M2	Public Portal & Inventory Engine	Weeks 3–4	Public website search, unit detail maps, row-level database locks for anti-double booking.
M3	Off-Plan Sales & Customer Portal	Weeks 5–6	CRM vault, milestone payment schedule generator, buyer self-service portal.
M4	AR/AP, Invoicing & Petty Cash	Weeks 7–8	Bulk invoicing, automated receipting, supplier invoice staging, multi-box petty cash module.
M5	Contract Lifecycle & Construction	Weeks 9–10	Unit swap engine, cancellation/refund approval workflow, construction milestone tracker.
M6	Reporting, Data Migration & Go-Live	Weeks 11–12	Financial P&L / Balance Sheet reports, Excel historical data import, penetration test, production launch.

2. Resource Allocation & Budget Projection

Role Title	Headcount / Allocation	Weekly Rate	Total Investment (12 Wks)
Chief Technology Officer / Lead Architect	1 FTE (100%)	$3,500 / week	$42,000
Senior Backend Engineers (NestJS/PostgreSQL)	2 FTE (100%)	$2,800 / engineer / wk	$67,200
Senior Frontend Engineers (React/TS)	2 FTE (100%)	$2,800 / engineer / wk	$67,200
QA Automation & Security Engineer	1 FTE (100%)	$2,000 / week	$24,000
DevOps & Cloud Engineer	1 FTE (100%)	$2,200 / week	$26,400
Cloud Infrastructure & Security Audit	AWS & Security Vendor	Flat allocation	$13,200
		TOTAL ESTIMATED INVESTMENT:	**$240,000 USD**